OBSCURE — The Builder's Browser

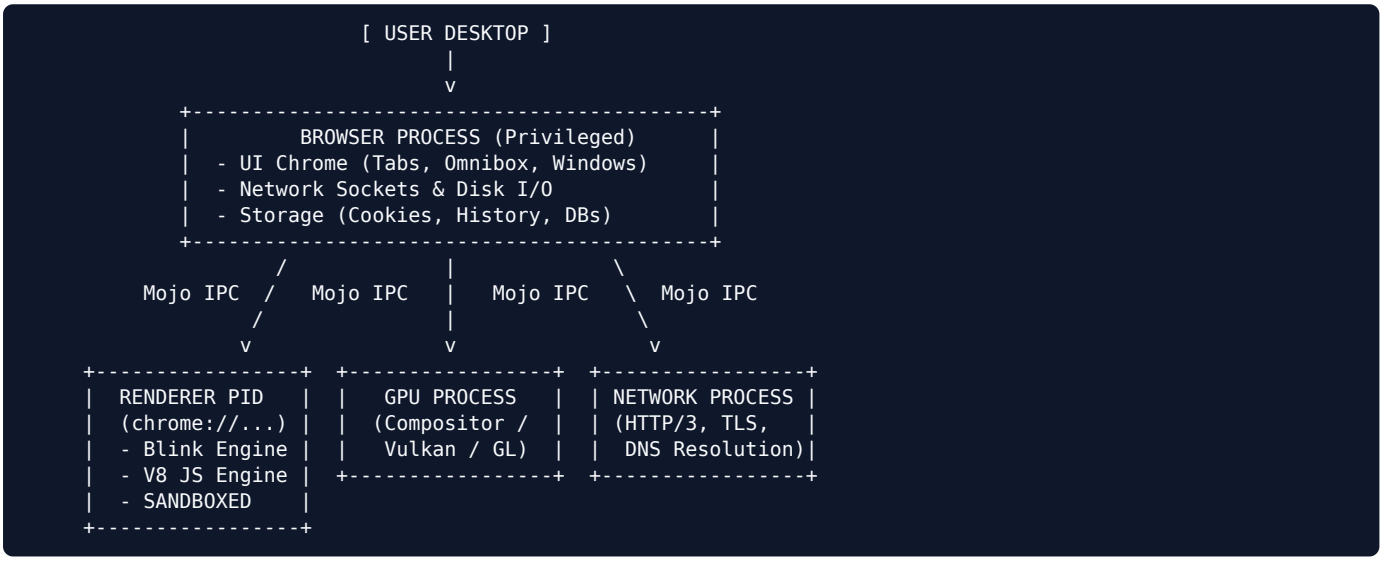
ZORQUELZ TECHNOLOGIES

Founder Mastery Engineering & Architecture Handbook

Target Audience:	Founder & CEO (Zorquelz)	**Control Branch:**	`feature/pr-1-qa-corrections`
Author / Lead:	Antigravity (Implementation Lead)	**Chromium Baseline:**	Stable `155.0.8059.39`
Chief Architect:	ChatGPT	**Document Purpose:**	Complete Founder Mastery & Technical Defense

1. Chapter 1: Chromium Multi-Process Architecture & Engine Internals

Chromium is designed as an **operating system inside an operating system** to prevent single-tab memory leaks and crashes from destabilizing the entire browser window.



- **Browser Process:** Privileged supervisor. Owns OS window handles, disk profile I/O, and raw network sockets.
- **Renderer Process:** Sandboxed worker. Runs Blink (HTML/CSS layout) and V8 (JavaScript execution). Has zero raw disk or socket access.
- **Zygote Process:** Pre-forked Linux template process used to spawn warm renderers instantly without full process startup overhead.
- **WebUI:** Chromium's native pipeline for building internal browser pages using web standards backed by C++ controllers.

2. Chapter 2: The "Shim Pattern" & Rebase Sustainability

> **The 10-Line Rule:** Large Chromium forks fail because they scatter modifications across core files. Obscure places >98% of its code in `//chrome/browser/obscure/` and restricts core diffs to ≤10 lines of registration hooks.

Hook ID	File & Location	Responsibility
HK-01	`chrome/browser/ui/webui/chrome_web_ui_configs.cc`	Registers `ObscureNewTabPageUIConfig` substituting upstream NTP on `chrome::kChromeUINewTabPageHost`.
HK-02	`chrome/browser/BUILD.gn`	Appends `"//chrome/browser/obscure"` to `source_set("browser")` public dependencies.

3. Chapter 3: Git Workflow, Bundles & Bulletproof Rollback Safety

Chromium contains over 40 million lines of code. Reckless commands can destroy weeks of work. Obscure operates under a four-tier rollback safety model:

1. **Tier 1 (Git Revert):** Creates inverse commits; non-destructive.
2. **Tier 2 (Patch Reversal):** `git apply --reverse` cleanly removes a patch series.
3. **Tier 3 (Hard Reset to Immutable Tag):** `git reset --hard obscure-baseline-155.0.8059.39` guarantees 100% clean baseline recovery.
4. **Tier 4 (Git Bundles):** `git bundle create backups/obscure.bundle ...` packages full commits into portable binary archives that restore identically even after hard resets.

> **Line Ending Invariant:** We enforce `*.patch -text` in `.gitattributes` so Windows Git never converts LF to CRLF, preventing `git am` patch failures inside WSL2.

4. Chapter 4: Script Tooling & Empirical QA Runners

Script	Mechanics & Flags	What It Establishes
`sync_sources.py`	Computes SHA-256 of native C++ and WebUI files; copies to WSL2 tree without symlinks.	Guarantees 100% deterministic code synchronization without cross-filesystem NTFS/ext4 index corruption.
`test_tb10_patch_revert.sh`	Creates synthetic patch, tests `git apply --check`, applies with `git am`, and resets to baseline tag.	Proves TB-10: patches apply cleanly and revert with 100% exact cryptographic hash match (`tb10_revert_test.log`).
`check_sandbox.py`	Inspects live `/proc/<pid>/status` and `/proc/<pid>/ns/` on active Chromium processes.	Confirms `Seccomp: 2` (BPF syscall filter) and `NoNewPrivs: 1` (no privilege escalation) on live renderers.
`demonstrate_rollback_recovery.sh`	Simulates custom Obscure work, creates Git bundle, hard resets to baseline, and pulls from bundle.	Empirically proves zero work loss: `BEFORE_COMMIT == AFTER_COMMIT` (`rollback_recovery_demonstration.log`).

5. Chapter 5: Founder Defensible Knowledge (Investor & Architect Q&A)

Q: Why did you build Obscure on Chromium rather than building a browser engine from scratch?

Answer: Building a modern web rendering engine from scratch requires handling decades of CSS, HTML5, WebGL, WebGPU, JavaScript specifications, and thousands of video/audio codecs. Even Microsoft gave up on EdgeHTML to adopt Chromium. Chromium is the de facto engine of the modern web. Obscure's value is not reinventing the wheel—it's taking Chromium's world-class engine and giving developers unprecedented security isolation, workspace intelligence, and deep network/port visibility.

Q: How do you guarantee user privacy in Incognito windows?

Answer: We enforce privacy at the C++ configuration layer. In `ObscureNewTabPageUIConfig::IsWebUIEnabled()`, we check `!profile->IsOffTheRecord()`. In Incognito and Guest modes, this returns false, causing Chromium to fall back to its native private screens. Obscure UI and custom state never load inside private sessions.